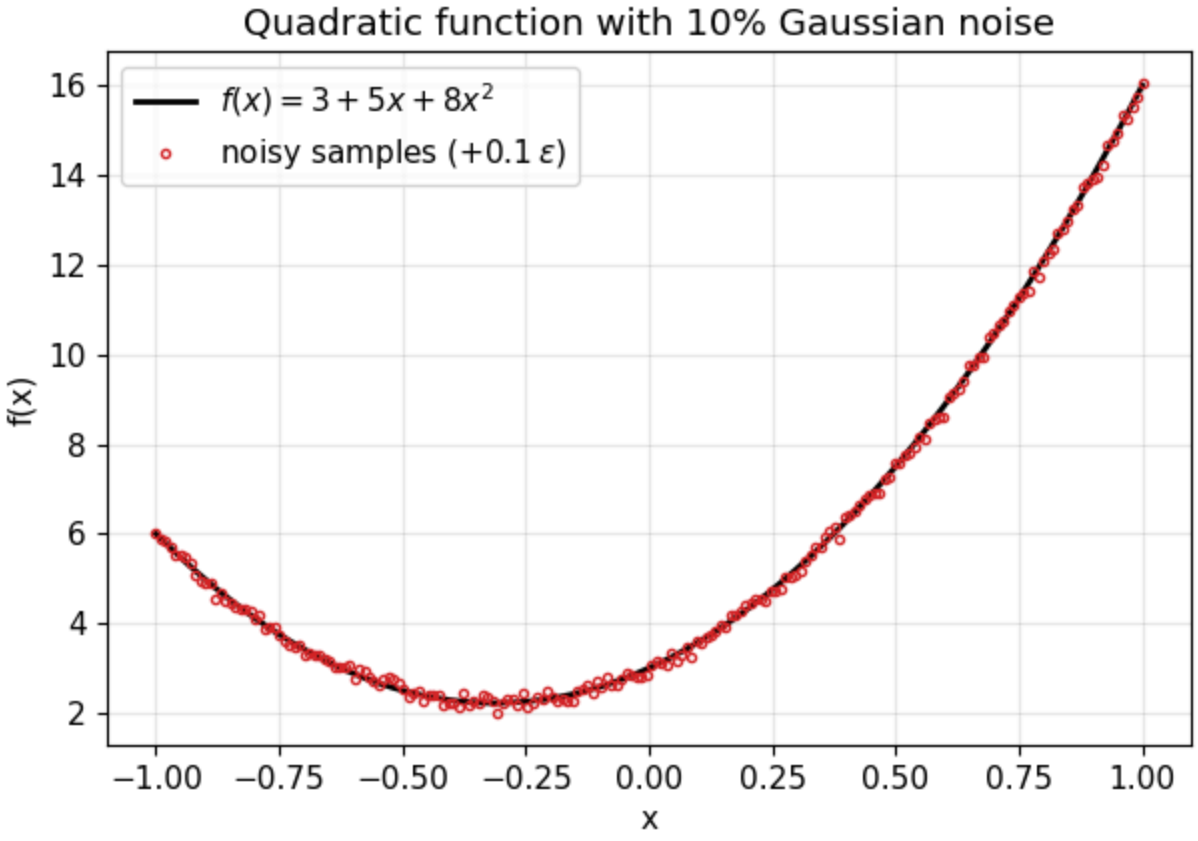
Homework 2 Solution

```
import numpy as np
import matplotlib.pyplot as plt
from PIL import Image
DATA = "hwk2_data/"
plt.rcParams.update({"figure.dpi": 100, "font.size": 11})
np.set_printoptions(precision=4, suppress=True)
```

Problem 1: Quadratic with 10% noise

We evaluate $f(x) = 3 + 5x + 8x^2$ on $[-1, 1]$ and add noise $0.1\,\varepsilon$ with $\varepsilon \sim \mathcal{N}(0, 1)$.

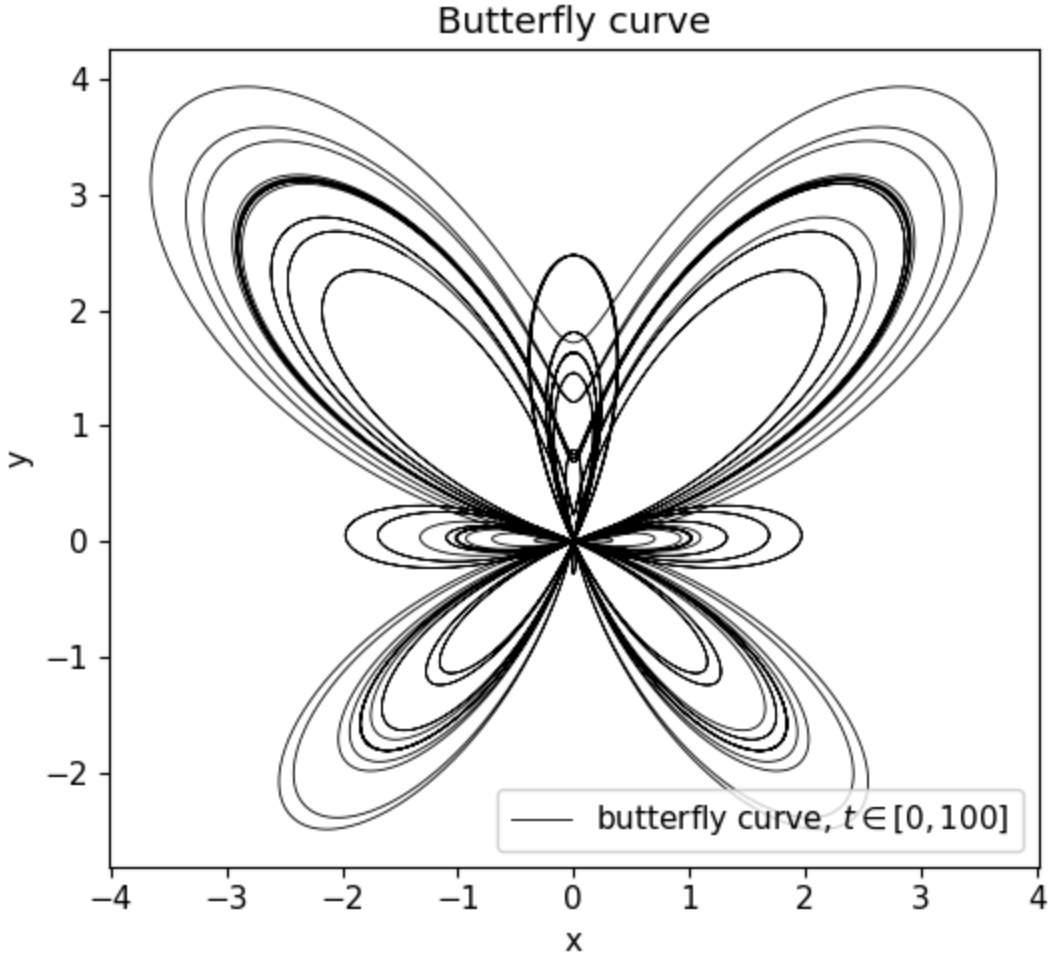
```
rng = np.random.default_rng(0)
x = np.linspace(-1, 1, 200)
f = 3 + 5*x + 8*x**2
y = f + 0.1*rng.standard_normal(x.size)
fig, ax = plt.subplots(figsize=(7,4.5))
ax.plot(x, f, "k-", lw=2, label=r"$f(x)=3+5x+8x^2$")
ax.plot(x, y, "o", ms=3, mfc="none", color="tab:red", label="noisy samples
($+0.1\\,\\varepsilon$)")
ax.set_xlabel("x"); ax.set_ylabel("f(x)"); ax.set_title("Quadratic function with 10%
Gaussian noise")
ax.legend(); ax.grid(alpha=.3); plt.show()
```



Problem 2: Butterfly curve

With $r(t) = e^{\cos t} - 2\cos(4t) - \sin^5(t/12)$, plot $x = r(t)\sin t$, $y = r(t)\cos t$ for $t \in [0, 100]$. Many points (20000) are needed to resolve the rapid oscillations.

```
t = np.linspace(0, 100, 20000)
r = np.exp(np.cos(t)) - 2*np.cos(4*t) - np.sin(t/12)**5
X, Y = r*np.sin(t), r*np.cos(t)
fig, ax = plt.subplots(figsize=(6,6))
ax.plot(X, Y, "k-", lw=0.6, label="butterfly curve, $t\\in[0,100]$")
ax.set_aspect("equal"); ax.set_xlabel("x"); ax.set_ylabel("y")
ax.set_title("Butterfly curve"); ax.legend(loc="lower right"); plt.show()
print("x range:", X.min(), X.max(), " y range:", Y.min(), Y.max())
```




```
x range: -3.652924762743516 3.652906987545444  y range: -2.487418610673017
3.929623169763543
```

Problem 3: Eigen-analysis of AB

```
A = np.array([[3,-1,2],[0,1,2],[1,0,-1]], float)
B = np.array([[3,0,1],[-1,1,0],[2,2,-1]], float)
AB = A@B
print("AB =\n", AB)
w, V = np.linalg.eig(AB)
print("eigenvalues:", w)
print("eigenvectors (columns):\n", V)
print("Gram matrix V^T V (identity iff orthonormal):\n", V.T@V)
print("AB symmetric?", np.allclose(AB, AB.T))
print("check AB V = V diag(w):", np.allclose(AB@V, V*w))
```

```
AB =
 [[14.  3.  1.]
 [ 3.  5. -2.]
 [ 1. -2.  2.]]
eigenvalues: [14.9198  0.5992  5.481 ]
eigenvectors (columns):
 [[-0.9584 -0.176   0.2246]
 [-0.2838  0.5042 -0.8156]
 [-0.0303  0.8455  0.5332]]
Gram matrix V^T V (identity iff orthonormal):
 [[ 1.  0. -0.]
 [ 0.  1. -0.]
 [-0. -0.  1.]]
AB symmetric? True
check AB V = V diag(w): True
```

Are the eigenvectors orthogonal?

Yes. Here $B = A^T$ (compare the entries), so $AB = AA^T$ is a real **symmetric** positive semi-definite matrix (printed above: `AB symmetric? True`). The spectral theorem guarantees that a real symmetric matrix has real eigenvalues and orthogonal eigenvectors for distinct eigenvalues. The eigenvalues are real, positive and distinct ($\approx 0.599, 5.481, 14.920$; positive because A is invertible, so AA^T is positive definite), and `np.linalg.eig` returns unit-norm eigenvectors whose Gram matrix V^TV equals the identity to machine precision, i.e. they form an orthonormal basis. (For a general non-symmetric product such as BA with $B \neq A^T$ this would not hold; note also that $BA = A^TA$ is a different symmetric matrix.)

Computing $AB - 2A^2 - I$

```
R = AB - 2*(A@A) - np.eye(3)
print("A^2 =\n", A@A)
print("AB - 2A^2 - I =\n", R)
```

```
A^2 =
 [[11. -4.  2.]
 [ 2.  1.  0.]
 [ 2. -1.  3.]]
AB - 2A^2 - I =
 [[-9. 11. -3.]
 [-1.  2. -2.]
 [-3.  0. -5.]]
```

The result is the matrix printed above; note that A^2 means the matrix product $A \cdot A$, not an elementwise square.

Problem 4: Low-rank approximation of the horse image

The image is converted to grayscale (luminance) so that a single SVD $M = U\Sigma V^T$ suffices. The rank-r approximation is $M_r = \sum_{i=1}^{r} \sigma_i u_i v_i^T$, which is the best rank-r approximation in the Frobenius norm (Eckart-Young). Storage for M_r is $r(m + n + 1)$ numbers versus mn.

```
img = Image.open("hwk2_data/horse.jpg")
print("original size/mode:", img.size, img.mode)
M = np.asarray(img.convert("L"), float)
mm, nn = M.shape
U, s, Vt = np.linalg.svd(M, full_matrices=False)
ranks = [10, 20, 120]
approx = {r: (U[:,:r]*s[:r])@Vt[:r] for r in ranks}
print("rank of image matrix:", len(s))
for r in ranks:
    err = np.linalg.norm(M-approx[r])/np.linalg.norm(M)
    store = r*(mm+nn+1)/(mm*nn)
    print(f"r={r:4d}  relative Frobenius error={err:.4f}  storage fraction={store:.3f}
energy kept={(s[:r]**2).sum()/(s**2).sum():.4f}")
```

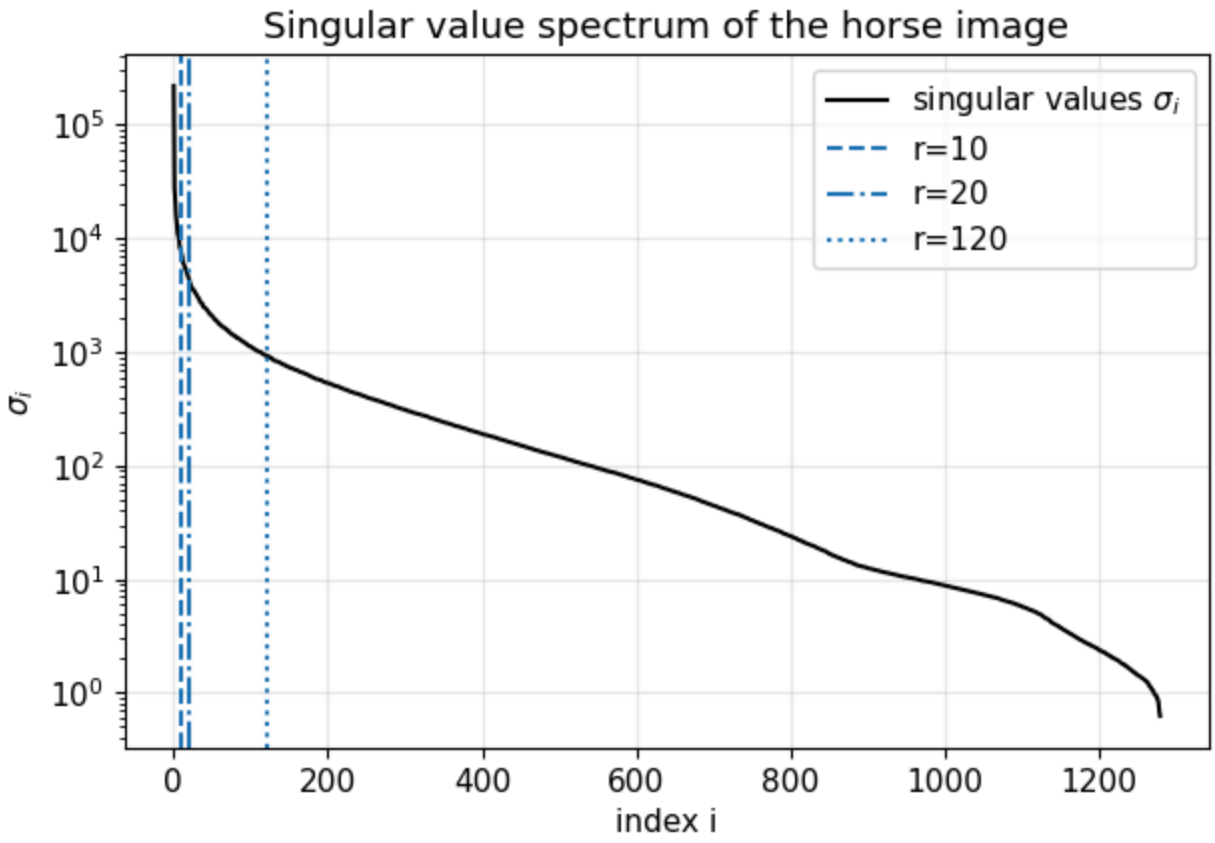
```
original size/mode: (1920, 1280) RGB
```

```
rank of image matrix: 1280
r=  10  relative Frobenius error=0.1251  storage fraction=0.013  energy kept=0.9843
r=  20  relative Frobenius error=0.0957  storage fraction=0.026  energy kept=0.9908
r= 120  relative Frobenius error=0.0363  storage fraction=0.156  energy kept=0.9987
```

```
fig, axs = plt.subplots(2, 2, figsize=(10,8))
for ax, (ttl, im) in zip(axs.ravel(), [("Original (grayscale)", M)] + [(f"Rank r={r}",
approx[r]) for r in ranks]):
    ax.imshow(np.clip(im,0,255), cmap="gray", vmin=0, vmax=255); ax.set_title(ttl);
ax.axis("off")
plt.tight_layout(); plt.show()
```




```
fig, ax = plt.subplots(figsize=(7,4.5))
ax.semilogy(s, "k-", label="singular values $\\sigma_i$")
for r, ls in zip(ranks, ["--", "-.", ":"]):
    ax.axvline(r, ls=ls, color="tab:blue", label=f"r={r}")
ax.set_xlabel("index i"); ax.set_ylabel(r"$\sigma_i$"); ax.set_title("Singular value
spectrum of the horse image")
ax.legend(); ax.grid(alpha=.3); plt.show()
```



Problem 5: Cubic fit via the SVD pseudo-inverse

Fit $y \approx c_0 + c_1 x + c_2 x^2 + c_3 x^3$. With Vandermonde matrix $X \in \mathbb{R}^{N\times 4}$ and economy SVD $X = U\Sigma V^T$, the least-squares solution is $c = X^+ y = V\Sigma^{-1}U^T y$.

```
xd = np.load(DATA+"xdata.npy"); yd = np.load(DATA+"ydata.npy")
print("shapes:", xd.shape, yd.shape, " x range:", xd.min(), xd.max())
X = np.vander(xd, 4, increasing=True)          # columns 1, x, x^2, x^3
U, s, Vt = np.linalg.svd(X, full_matrices=False)
c_svd = Vt.T @ ((U.T @ yd)/s)
c_pf = np.polyfit(xd, yd, 3)[::-1]               # reorder to c0..c3
print("singular values of X:", s, " cond:", s[0]/s[-1])
print("SVD coefficients  c0..c3:", c_svd)
print("np.polyfit        c0..c3:", c_pf)
print("max abs difference:", np.abs(c_svd-c_pf).max())
res = yd - X@c_svd
print("RMSE:", np.sqrt(np.mean(res**2)))
```

```
shapes: (100,) (100,)  x range: -2.0 2.0
singular values of X: [32.9704 19.8899  6.1161  4.4064]  cond: 7.482460438436073
SVD coefficients  c0..c3: [ 1.8338 -0.5752  0.2458  0.8406]
np.polyfit        c0..c3: [ 1.8338 -0.5752  0.2458  0.8406]
max abs difference: 7.66053886991358e-15
RMSE: 0.8788368445207191
```

```
xx = np.linspace(xd.min(), xd.max(), 400)
fig, ax = plt.subplots(figsize=(7,4.5))
ax.plot(xd, yd, "ko", ms=4, mfc="none", label="data")
ax.plot(xx, np.vander(xx,4,increasing=True)@c_svd, "r-", lw=2, label="cubic fit (SVD
pseudo-inverse)")
ax.plot(xx, np.polyval(c_pf[::-1], xx), "b--", lw=2, label="np.polyfit")
ax.set_xlabel("x"); ax.set_ylabel("y"); ax.set_title("Cubic polynomial fit by SVD")
ax.legend(); ax.grid(alpha=.3); plt.show()
```

